
Reproducible projects in R:

practical workshop, part 1

Dr Marina Vabistsevit

Research Fellow in Clinical and Biomedical Sciences
UK Reproducibility Network (UKRN) Local Network Lead for Exeter

What this workshop is about

Part 1: (today!)

Project-oriented workflow:

- Organising your projects with .Rproj
- Using renv to manage packages in your Rstudio environment
- [Practical part]
- Reproducibility habits and efficiency tips

Part 2: (next Friday)

Using Git in your R project



Icon for R



Icon for RStudio



software
company who
develop
Rstudio

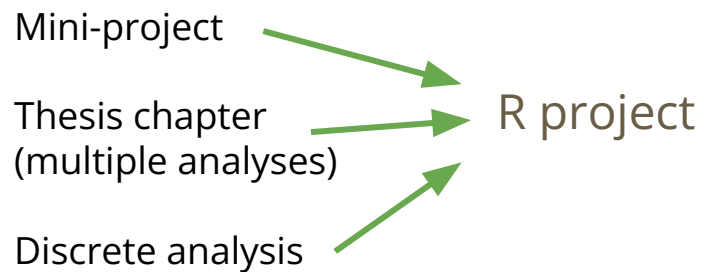


Positron -
new IDE from
Posit for
R/Python etc

The slides will be shared after!

Project-oriented workflow

R projects



R projects



R projects



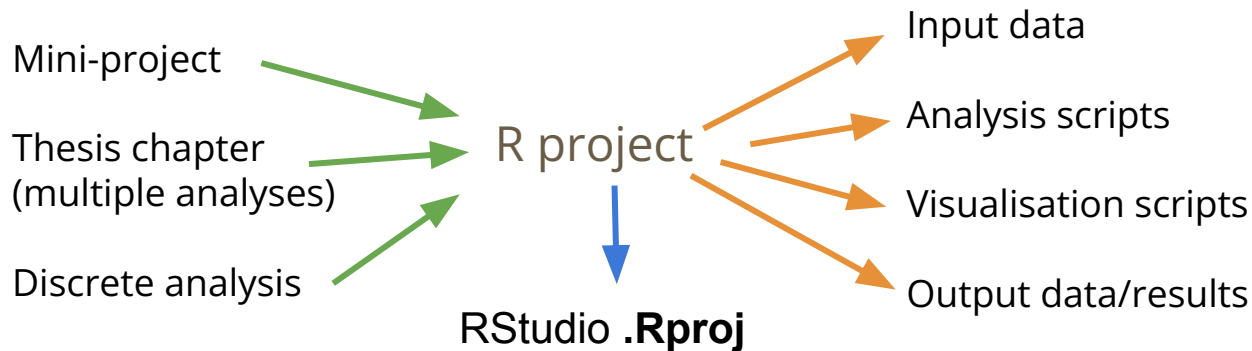
How you do organise your projects? scripts? folders?

Where do you store and access your data? results? figures?

Does it matter? Are your analysis/projects reproducible?

Can you organise your projects better and make life easier for future self/colleagues?

R projects



How you do organise your projects? scripts? folders?

Where do you store and access your data? results? figures?

Does it matter? Are your analysis/projects reproducible?

Can you organise your projects better and make life easier for future self/colleagues?

Reproducibility can be enhanced through intentionally organising projects with .Rproj, i.e. working in **project-oriented workflow**

Project-oriented workflow



Photo by secumem

📅 2017/12/12

👤 Jenny Bryan

I was honored to speak this week at the IASC-ARS/NZSA Conference, hosted by the Stats Department at The University of Auckland. One of the conference themes is to celebrate the accomplishments of Ross Ihaka, who got R started back in 1992, along with Robert Gentleman. My talk included advice on setting up your R life to maximize effectiveness and reduce frustration.

Two specific slides generated much discussion and consternation in #rstats Twitter:

If the first line of your R script is

```
setwd("C:\\Users\\jenny\\path\\that\\only\\I\\have")
```

I will come into your office and SET YOUR COMPUTER ON FIRE 🔥.

If the first line of your R script is

```
rm(list = ls())
```

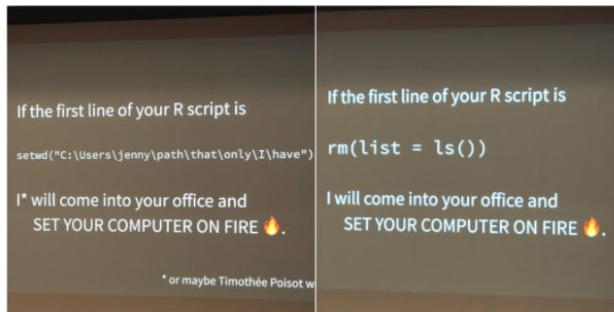
I will come into your office and SET YOUR COMPUTER ON FIRE 🔥.

Twitter profile card for Jenny Bryan (@JennyBryan). It includes her profile picture, a 'Following' button, and a bio: 'Software engineer @rstudio, humane #rstats, adjunct prof @UBC where I created @STAT545, part of @ropensci, she/her'. Location is 'Vancouver, BC' and website is 'jennybryan.org'.

Hadley Wickham
@hadleywickham

Follow

The only two things that make @JennyBryan 🤔😡💩. Instead use projects + here::here() #rstats



4:50 PM - 10 Dec 2017

290 Retweets 950 Likes



Why setting directory does not work

```
setwd("path/that/only/works/on/my/machine")
```

- Only works on your system
- Unlikely to work in a few years or on another computer
- Relying on hard-coded paths make a project easy to break and potentially not reproducible (shared data paths on a server is a different matter)

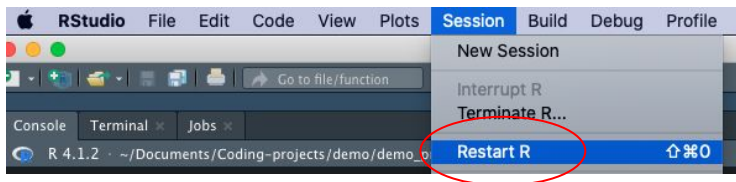
→ **Don't use `setwd()` to set your current path / directory**

Save your source code, not the workspace

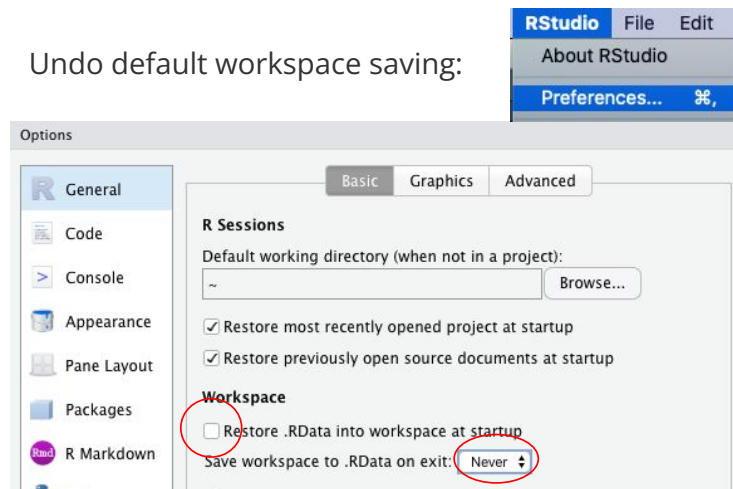
Save important objects and intermediates to files and scripts, in a modular way

If there is no source, it's not reproducible

Avoid relying on objects in your workspace and **restart R daily*** to ensure a clean environment:



Undo default workspace saving:



→ Do not use `rm(list = ls())` to clean your Rstudio env

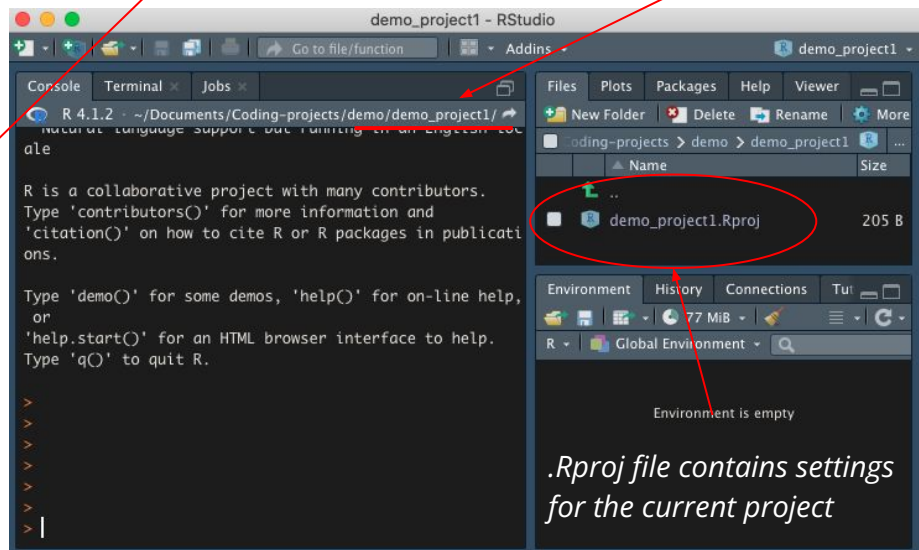
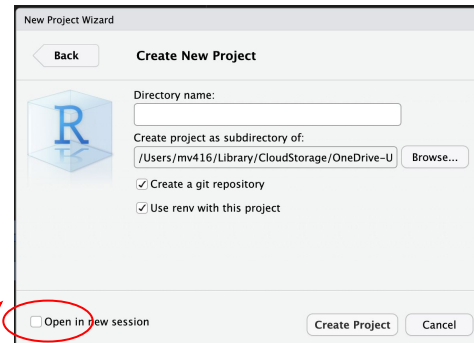
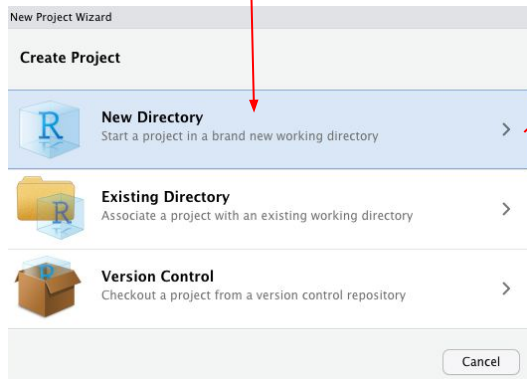
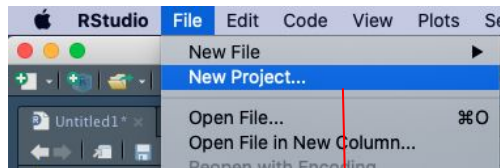
*whatever is reasonable for you

To solution to both

- Having to set your working directory path
 - Cleaning your workspace environment
- **organising your work as an .Rproj (i.e. project-oriented workflow) helps you manage your workspace env and file paths**

Project-oriented workflow

Creating your new project in Rstudio as .Rproj:



Project directory

.Rproj file contains settings
for the current project

This means that you are essentially compartmentalizing your current project

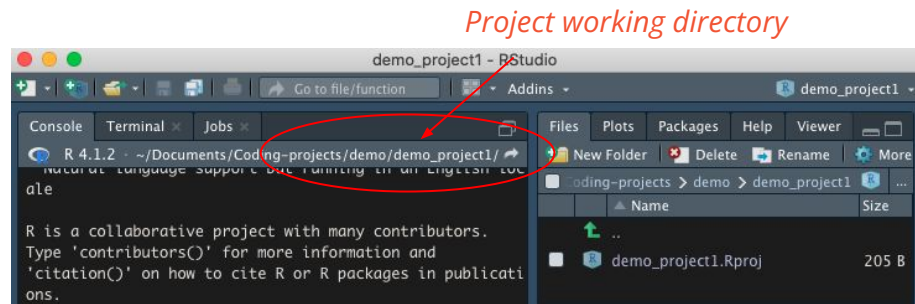
How working within .Rproj improves reproducibility:

File system discipline: project directory stores all your data, scripts, figures

File path discipline: The working directory is set to the project directory (e.g.

demo_project1/), so you don't need to *setwd()* or specify full paths to data (only internal subfolders that are relative to top directory)

Working directory intentionality: when working on project A, your working directory is set to project A's folder



- The project **creates everything it needs**, within its workspace/folder, and **touches nothing it did not create**
- Any scripts are written assuming they will be **run from a fresh R session** within the project
- The project folder **can be moved anywhere**, and everything will still work (no paths will be broken)

You don't need to use setwd()

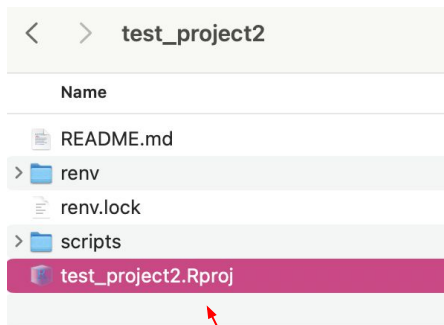
Keeping your work as an **.Rproj** will help you manage your file paths
(all subfolder paths are relative to directory root, i.e. Rproj location):

```
(base) [demo_project1] tree -L 3
.
├── README.md
├── data
│   ├── processed
│   │   └── data_file_clean.tsv
│   └── raw
│       └── raw_data_file.tsv
├── demo_project1.Rproj
├── figures
├── results
├── scripts
│   ├── 01_load_raw_data_and_clean.R
│   ├── 02_explore_data.R
│   ├── 03_run_analysis.R
│   ├── 04_make_plots.R
│   └── 05_project_summary.Rmd
```

```
01_load_raw_data_and_clean.R
1 library(readr)
2
3 # read raw_data_file.tsv
4 data_raw <- read_tsv("data/raw/raw_data_file.tsv")
5
6 # clean data
7 # <cleaning code>
8 data_clean <- data_raw
9
10 # save clean data
11 write_tsv(data_clean, "data/processed/data_file_clean.tsv")
12
13
14
```

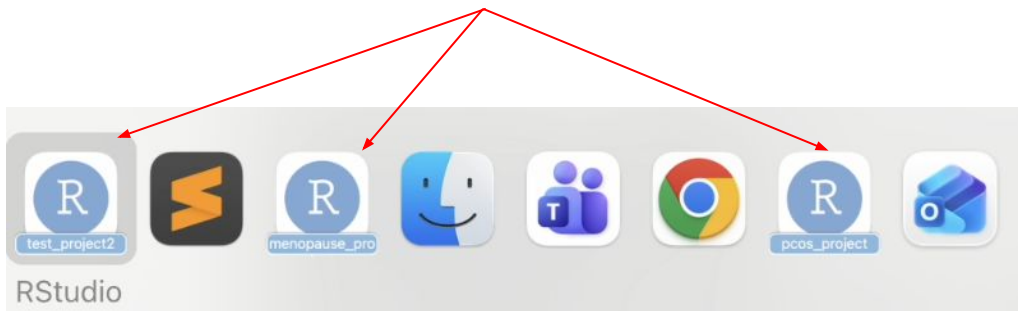
You don't need to use `rm(list=ls())`

Click on `.Rproj` file in your project directory to open a new clean session in your project:



Simply click on `.Rproj` to open a project-specific Rstudio session

You can also have multiple non-conflicting projects open at the same time:



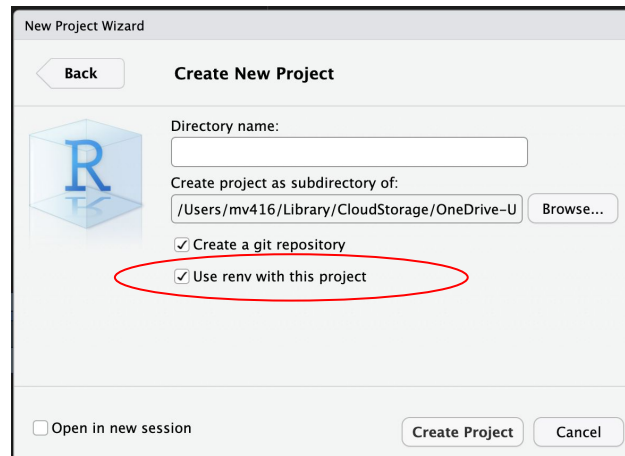
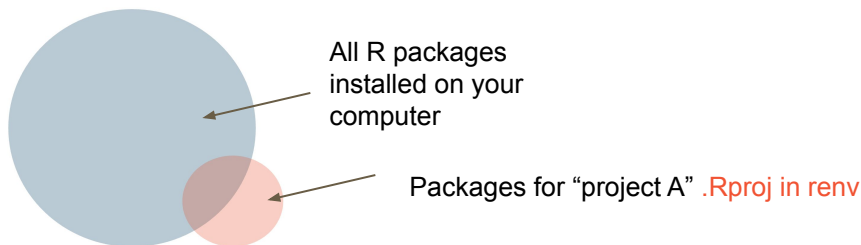
R environments

renv for managing package versions within projects

renv package helps you create reproducible environments for your R projects



- Keeps track of all your installed packages
- Installs them in a new location (i.e. easy to share)
- Allows to come back to an old project without code breaking due to updates in package versions



Setting up your renv

```
install.packages("renv")
```

`renv::init()` – initialise your project env

This creates:

NB this is the same as

☒ Use renv with this project

- **renv/**: stores packages for the project.
- **renv.lock**: records packages and the exact versions used
- **.Rprofile**: ensures renv activates when the project opens

☐	 .Rprofile	26 B
☐	 renv	
☐	 renv.lock	402 B
☐	 test_project1.Rproj	205 B

When renv is initialised, your project gets its own project library - a private folder (**renv/library/**) where packages are installed just for this project.

This means package versions are isolated and recorded for each project, so updating a package in one project won't break others.

This makes collaboration and reproducibility much easier.

renv::snapshot() – update your env with newly installed packages (saves to renv.lock)

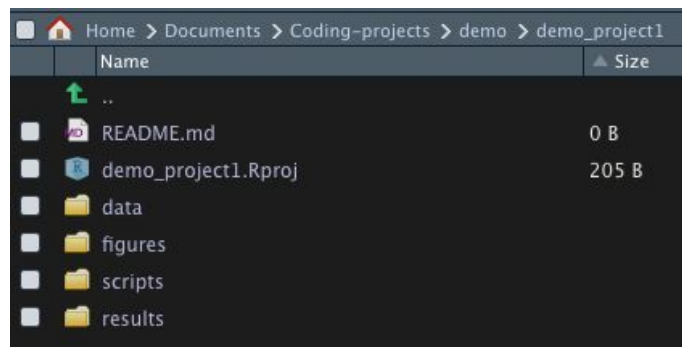
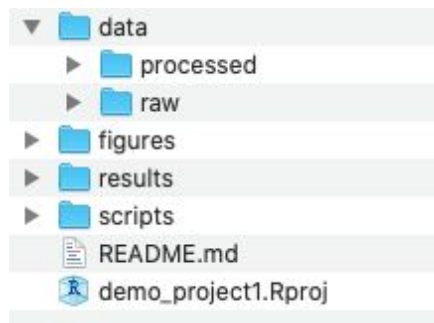
renv::restore() – when project is re-opened, load all your required packages

Practical part (15 mins)

1. **In your Rstudio, create a new Project (e.g. *test_project1*) with *renv* enabled in a new session (window) [if you have other work ongoing in Rstudio, keep it open in other window!]**
 - a. Create a new R script in the project folder; install package “sp” in it
 - b. Check the “sp” package loads in your project [*library(sp)*], then *renv::snapshot()* it
 - c. Check the “sp” package loads in your other ongoing open Rstudio session
 - d. Close your new project Rstudio window
 - e. Re-open by clicking on *test_project1.Rproj* file
 - f. Test the “sp” package still loads (do not install again!)
2. **Download ‘demo_project1.zip’ from <https://tinyurl.com/4czpk3cb>**
 - a. Open the project via *demo_project1.Rproj* and explore the folders structure
 - b. Has the project been created with *renv*? Check available packages in *renv/library/*
 - c. Copy a readable text file from your own work to *data/raw/* and read it in script 01
 - d. Close the project window; move the project folder elsewhere (eg Desktop); open the project from new location and try reading the file again in script 01. Did it work?

Reproducibility habits and other efficiency tips

Organise your projects intentionally

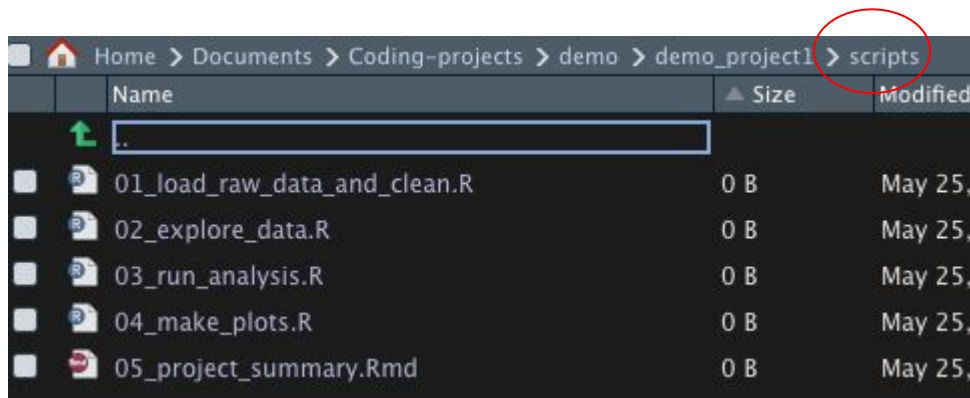


File system discipline

Take advantage of default ordering

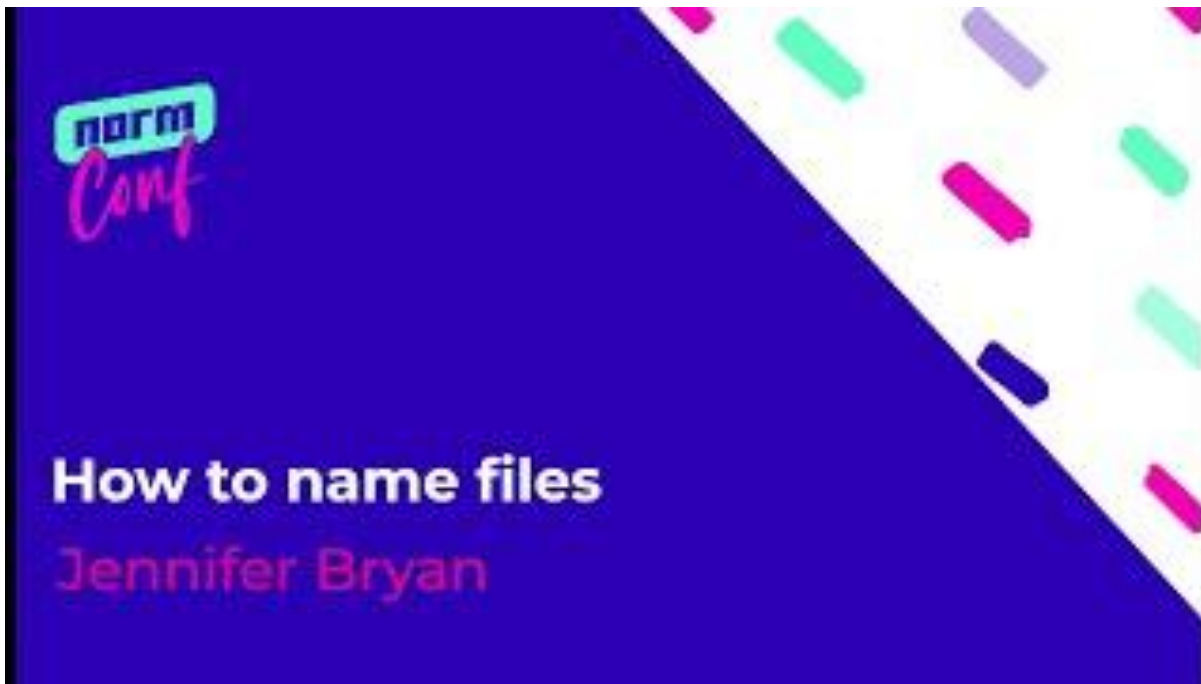
- 01_load_raw_data_and_clean.R
- 02_explore_data.R
- 03_run_analysis.R
- 04_make_plots.R
- 05_project_summary.Rmd

Can have many parts of the analysis separately - save interim results as files and re-read then in the next script

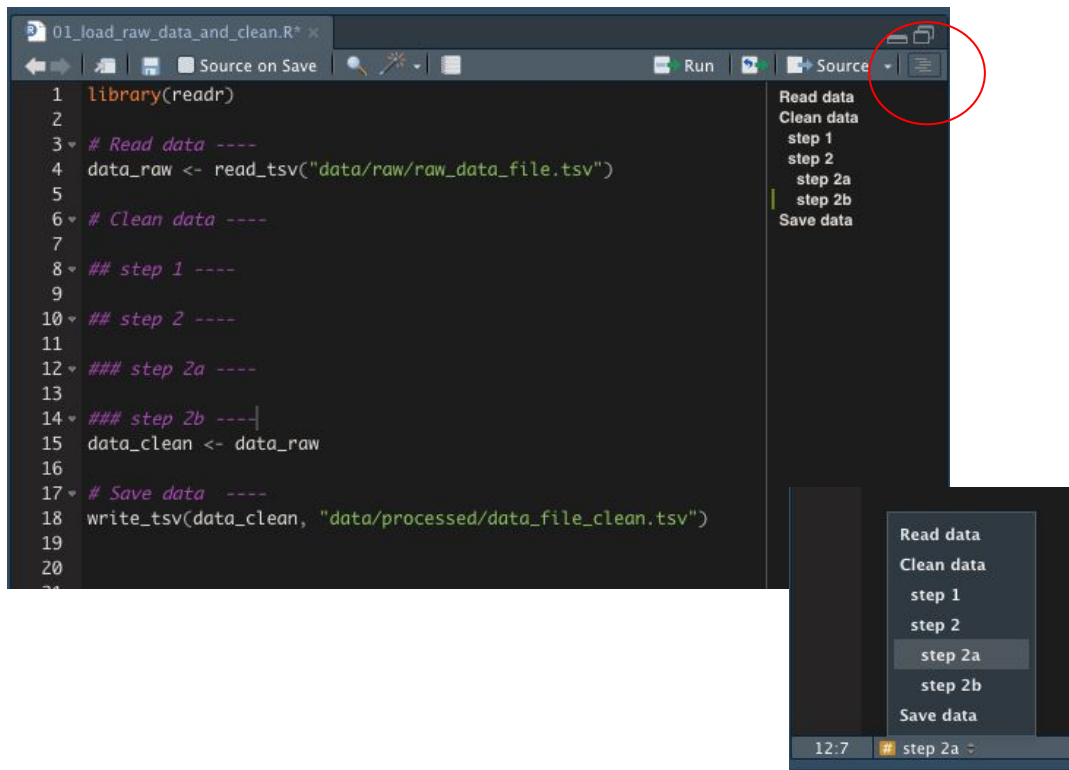


Name your files well

“How to name files” by Jenny Bryan - [link](#) (<5 mins)



Name your code sections and use them for quick navigation



- Use section headings:

section ----

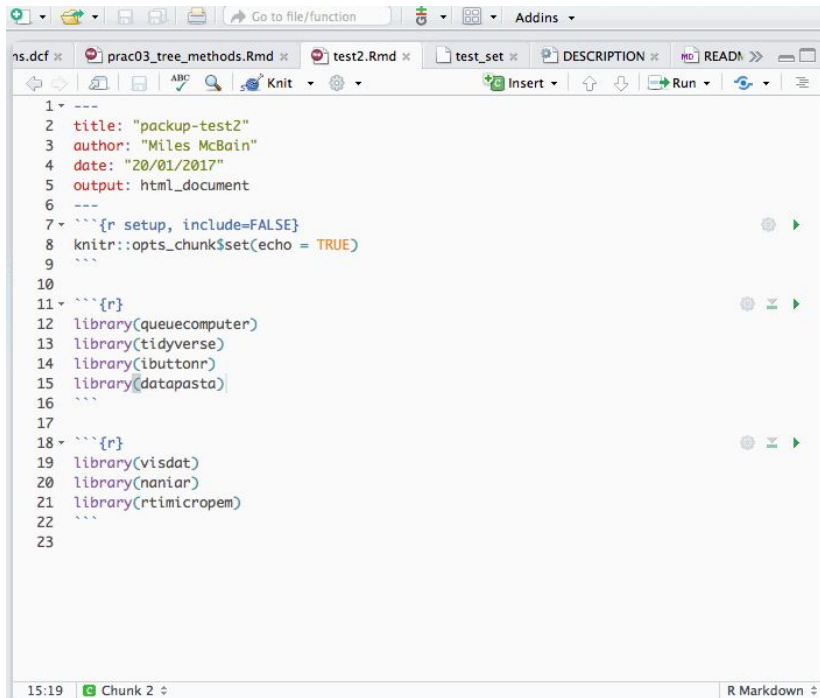
subsection ----

subsubsection ----

- Great for navigating in long scripts
- Can fold sections:



Keep all used libraries at the beginning of the script



```
1 ---
2 title: "packup-test2"
3 author: "Miles McBain"
4 date: "20/01/2017"
5 output: html_document
6 ---
7 ```{r setup, include=FALSE}
8 knitr::opts_chunk$set(echo = TRUE)
9 ```
10
11 ```{r}
12 library(queuecomputer)
13 library(tidyverse)
14 library(ibuttonr)
15 library(datapasta)
16 ```
17
18 ```{r}
19 library(visdat)
20 library(naniar)
21 library(rtimicropem)
22 ```
23
```

packup R add-in helps to collect and move all used libraries to the top

To install *packup* add-in :

```
devtools::install_github("milesmbain/packup")
```

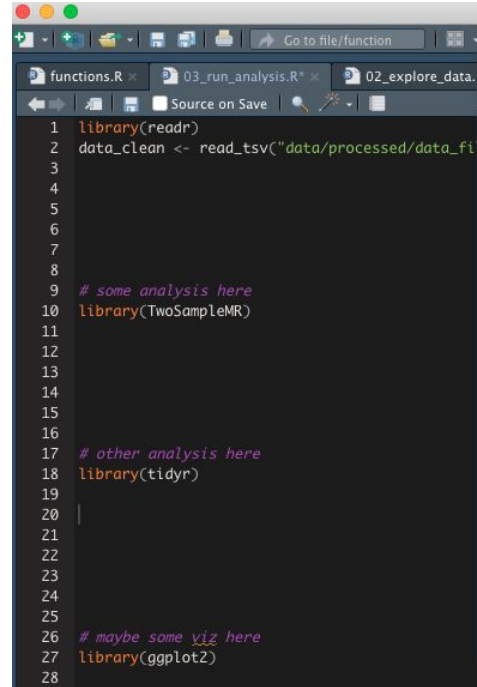
<https://github.com/MilesMcBain/packup>

Move all libraries to the top

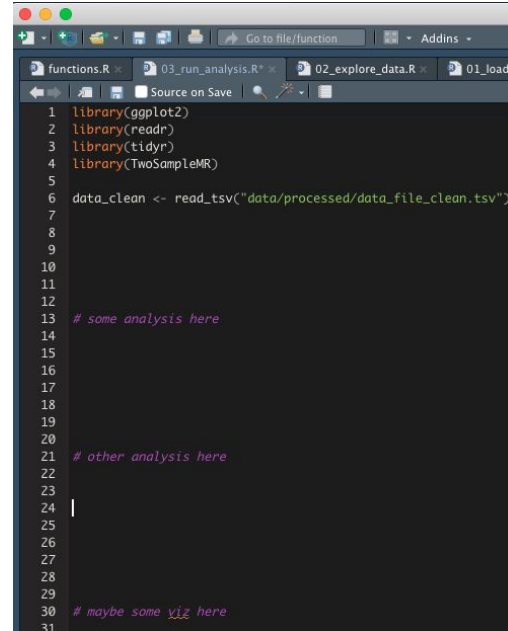
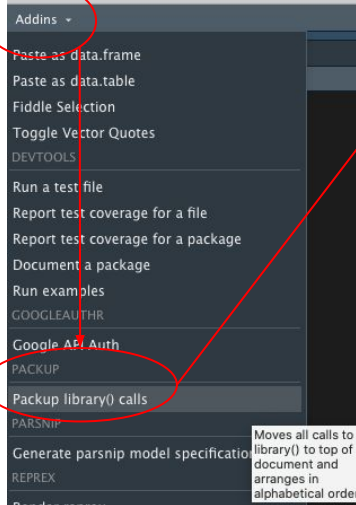
Install *packup* add-in:

```
devtools::install_github("milesmbain/packup")
```

Call it from Addins menu:



```
1 library(readr)
2 data_clean <- read_tsv("data/processed/data_file_clean.tsv")
3
4
5
6
7
8
9 # some analysis here
10 library(TwoSampleMR)
11
12
13
14
15
16
17 # other analysis here
18 library(tidyr)
19
20
21
22
23
24
25
26 # maybe some viz here
27 library(ggplot2)
28
```

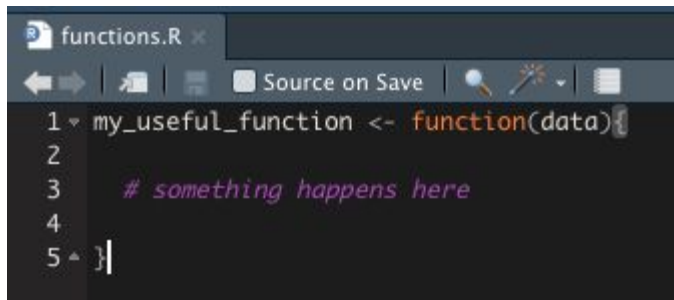


```
1 library(ggplot2)
2 library(readr)
3 library(tidyr)
4 library(TwoSampleMR)
5
6 data_clean <- read_tsv("data/processed/data_file_clean.tsv")
7
8
9
10
11
12
13 # some analysis here
14
15
16
17
18
19
20
21 # other analysis here
22
23
24
25
26
27
28
29
30 # maybe some viz here
31
```

Try this is script 03_run_analysis.R in demo_project1

Jump to function definitions or open data frame with cmd+click

```
## step 2 ----  
  
output <- my_useful_function(input)
```



The screenshot shows an RStudio editor window with a file named 'functions.R'. The code in the editor is as follows:

```
1 my_useful_function <- function(data){  
2  
3   # something happens here  
4  
5 }
```

Cmd + click on the function name:

- navigate to the function definition
- can be your own function, or a function from a packages
- this allows to review internal code / required arguments

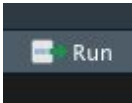
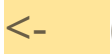
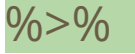
Cmd + click on data frame:

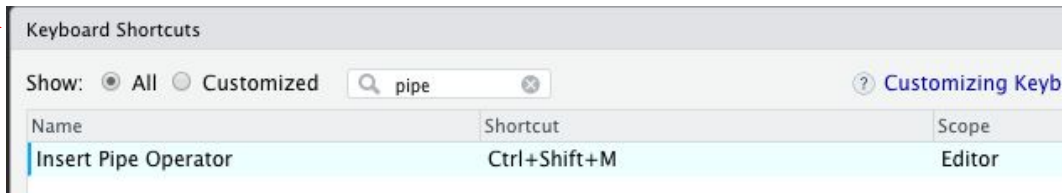
- opens the data frame in preview (i.e. same as `View()` , but without typing)

(Ctrl on Windows)

Keyboard shortcuts

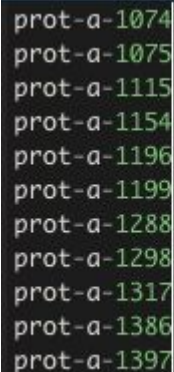


-  (option + Enter)
-  (option/alt + " - ")
-  (control + shift + M)
-  (control + shift + I)



Vertical selection

(hold *option* or *alt* and drag cursor down to select vertically)



```
prot-a-1074  
prot-a-1075  
prot-a-1115  
prot-a-1154  
prot-a-1196  
prot-a-1199  
prot-a-1288  
prot-a-1298  
prot-a-1317  
prot-a-1386  
prot-a-1397
```

Great for e.g.

- commenting out a block of code with #
- adding "" around a column of ids

Any other R tips to share?

End of part 1

Summary: using .Rproj for organising work

- “Reproducible project in R” means:
 - **File system discipline:** all files related to a single project are stored in a designated folder;
 - **Working directory discipline:** intentionally work in project directory when opening Rproj
 - **File path discipline:** paths are relative to the project directory (not hard-coded full paths!)
 - **Daily work habit:** Restarting R often and re-running your script under development from the top will help you catch issues early on
 - **Using git for version control (next time!)**
- Practising these habits together will give you the biggest pay-off
 - Reproducing your analyses will be easy
 - Organising your projects will help you make sense of them in 6/12/etc months
 - Can move your project anywhere or share it with anyone without changing paths

Final thoughts / disclaimers

- Project-oriented workflow in R is not suitable/applicable to every scenario
 - Sometimes data is stored externally, and can't moved/copied (so you can't use within-project paths, but potentially can use symbolic links)
 - Not all work is done interactively in Rstudio
 - Some people use R from the terminal on the server - again, because of data access/size
 - Some analyses are computation-heavy and require to be submitted as scripts / run in parallel on server
- ★ If your current workflow with `setwd()` works for you and your colleagues, that's totally fine, but consider future-proofing! ;)

Recommended and used resources

<https://www.tidyverse.org/blog/2017/12/workflow-vs-script/>

<https://richpauloo.github.io/2018-10-17-How-to-keep-your-R-projects-organized/>

<https://www.rforecology.com/post/organizing-your-r-studio-projects/>

<https://kkulma.github.io/2018-03-18-Prime-Hints-for-Running-a-data-project-in-R/>

<https://rstats.wtf/project-oriented-workflow.html>

<https://appsilon.com/rstudio-shortcuts-and-tips/>

<https://datacornering.com/my-favorite-rstudio-tips-and-tricks/>

<https://happygitwithr.com/>

<https://rstudio.github.io/renv/articles/renv.html>

<https://rstats.wtf/>

Thank you!



University
of Exeter



Promoting Open and Reproducible Research at Exeter

University of Exeter UK Reproducibility Network (UKRN)

The University of Exeter is a member of [UK Reproducibility Network](https://www.ukrn.ac.uk/) (UKRN).

UKRN functions as a nationwide consortium driven by peers, with the overarching goal of maintaining the UK's position as a hub for cutting-edge research. This objective is achieved through an exploration of the elements that bolster resilient research, the facilitation of training initiatives, and the widespread sharing of exemplary methodologies. Through collaborative partnerships with external stakeholders, the UKRN ensures seamless alignment of endeavors within the sector.

**JOIN THE
UKRN EXETER
MAILING LIST**



Stay informed
on upcoming
event and
initiatives



<https://www.exeter.ac.uk/research/openresearch/reproducibility/ukrn/>